GraphWeave — Complete Development Plan

GraphRAG for Multi-Hop Questions

Version 1.0 | 27 October 2026 | Engineering specification for an independently buildable portfolio project

Document control: Treat this document as the project-specific source of truth. If implementation choices change, update the relevant decision record and tests.

Delivery approach

Suggested sequence: one phase at a time, with a working main branch at each checkpoint. The time estimates below are planning estimates for one developer and will vary with provider setup and available time.

Phase-by-phase plan

1. Phase 0 — Dataset and graph schema

Define multi-hop question categories, ontology, provenance format, evaluation set, and baseline retrieval protocol.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

2. Phase 1 — Foundation and ingestion

Create FastAPI, PostgreSQL, Neo4j connectivity, document parsing, chunking, deterministic IDs, and ingestion job tracking.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

3. Phase 2 — Extraction pipeline

Implement schema-constrained entity/relation extraction, validation, confidence thresholds, retries, and provenance persistence.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

4. Phase 3 — Entity resolution and graph writes

Implement canonical IDs, aliases, conservative resolution, typed edges, versioning, and idempotent graph updates.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

5. Phase 4 — Vector and graph retrieval

Implement vector seed retrieval, bounded graph expansion, evidence fusion, deduplication, and trace logging.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

6. Phase 5 — Answer generation and UI

Generate citation-grounded answers and build document upload, query, answer evidence, and graph exploration views.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

7. Phase 6 — Evaluation and tuning

Compare vector-only, graph-only, and hybrid approaches; tune depth/fan-out and report quality, latency, and cost.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

8. Phase 7 — Hardening and delivery

Add tenant/corpus scope, query limits, deletion/re-ingestion behavior, Docker Compose, docs, sample corpus, and demo.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

Suggested schedule

- Week 1: Phase 0–1, architecture decisions, repo setup, database, API skeleton, and local environment.
- Week 2: Core data pipeline and baseline end-to-end workflow.
- Week 3: Project-specific core capability and failure handling.
- Week 4: Frontend dashboard/inspection UI and telemetry.
- Week 5: Security tests, evaluation benchmark, tuning, and documentation.
- Week 6: Packaging, demo data, final QA, screenshots, architecture diagram, and portfolio write-up.

Milestones and evidence

- M1 — Architecture approved: diagrams, data model, API contracts, security assumptions, and decision log.
- M2 — Vertical slice: one document ingested and one question answered end to end.
- M3 — Core differentiator: feature implemented with unit and integration tests.
- M4 — Observable application: dashboard/trace shows relevant metrics for each request.
- M5 — Evaluation: baseline and enhanced system measured on the same versioned dataset.
- M6 — Release candidate: automated tests pass, Docker startup is reproducible, documentation complete.

Risk register

- LLM extraction can create false relationships; preserve provenance, confidence, validation, and evaluation.
- Entity resolution can merge distinct entities; ambiguous matches should remain separate or require review.
- Graph expansion can explode in size; enforce traversal depth, fan-out, timeouts, and query budgets.
- Graph answers can sound certain despite weak paths; require evidence-backed citations and explicit uncertainty.

Release checklist

- Clean clone and setup verified from README.
- No committed credentials, local database files, or user data.
- Migrations, seed script, lint, type checks, tests, and production frontend build pass.
- Critical security boundaries have explicit tests.
- Evaluation report includes dataset description, method, baseline, results, limitations, and reproducibility details.
- Demo does not expose real secrets or sensitive documents.

- Resume claims are updated only after measured results exist; never invent percentages or claim unverified improvements.